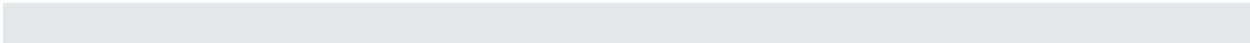


DEPARTMENT OF COMPUTER SCIENCE
Open Source Tool Presentation



Cypress

Open-Source End-to-End Web Testing Framework
A Comprehensive Technical Presentation & Demo Report



14 Total Tests Written	3 Testing Methodologies	4 Test Files Created
--	---	--

<i>Presented By</i> Ashley Almeida	Jaden Britto	Naomi Countino
---	--------------	----------------

Course Open Source Tools	Submission Type Technical Report	Status Complete — All Tests Passing	Version 1.0 — Final
-----------------------------	-------------------------------------	--	------------------------

TABLE OF CONTENTS

1. Abstract	3
2. Introduction.....	3
3. Background & Problem Statement	4
4. What is Cypress?	4
5. Project Overview	5
6. Core Cypress Commands.....	6
7. Detailed Test Analysis.....	7
8. Key Technical Concepts	9
9. Results & Discussion	10
10. Conclusion.....	10
11. References	11

1. Abstract

This report presents a comprehensive technical study of Cypress, a modern open-source end-to-end testing framework for web applications. As part of the Open Source Tools course, a fully functional Student Attendance Management System was developed using Next.js, serving as a live demonstration platform. Fourteen automated tests were authored across four distinct test files, covering three categories of software testing: End-to-End (E2E) testing, API testing, and Component testing. This document details the motivation for choosing Cypress, the architecture of the demo application, an in-depth analysis of each test file, key technical concepts employed, and a critical evaluation of Cypress as a testing solution for modern web development.

2. Introduction

In the modern landscape of web application development, ensuring software quality through automated testing has become a non-negotiable practice. As web applications grow in complexity, the cost of manual testing scales dramatically — making automated test frameworks an essential component of any professional development workflow.

This presentation introduces Cypress, an open-source testing framework that has rapidly gained adoption among developers and quality assurance engineers worldwide. Unlike legacy tools such as Selenium WebDriver, Cypress was engineered from the ground up for the modern JavaScript ecosystem. It operates directly within the browser, enabling real-time interaction, accurate simulation of user behaviour, and deterministic test execution.

The objective of this report is threefold:

1. To provide a thorough understanding of the Cypress framework — its architecture, design philosophy, and capabilities.
2. To demonstrate its practical application through a live web application built specifically for this purpose.
3. To analyse fourteen automated tests across three testing methodologies: End-to-End testing, API testing, and React Component testing.

The demo application — a Student Attendance Management System — serves as a realistic, real-world context that allows meaningful exploration of all three testing paradigms within a single cohesive project.

3. Background & Problem Statement

3.1 The Challenge of Manual Testing

Consider a standard web application feature: a login page. Every time a developer modifies the codebase — fixing a bug, updating the UI, or refactoring a function — the login functionality must be re-verified. In a manual testing environment, this requires a human tester to:

- Open the browser and navigate to the login page
- Enter valid credentials and verify the successful redirect
- Enter invalid credentials and verify the error message
- Submit an empty form and verify the validation response
- Repeat for every role (student, administrator) across every supported browser

Multiplied across fifty or more features in a typical production application, manual testing becomes both a bottleneck and a source of regression risk. Defects introduced in one area may silently break another, remaining undetected until they reach end users.

3.2 Why Automated Testing?

Automated testing resolves this challenge by encoding test scenarios as executable scripts. These scripts run in seconds, can be triggered on every code change, and produce consistent, repeatable results. The primary benefits include:

- Regression prevention — automatically catching previously working features that break
- Developer confidence — enabling safe refactoring without fear of unknown side effects
- Documentation — tests serve as living documentation of expected application behaviour
- Speed — an entire suite of hundreds of tests can execute in minutes

3.3 Why Cypress?

Cypress vs. Traditional Tools

Legacy E2E testing tools such as Selenium operate outside the browser, communicating via a WebDriver protocol. This introduces timing issues, network latency, and environmental flakiness. Cypress takes a fundamentally different approach — it executes tests inside the browser's JavaScript runtime alongside the application itself.

This architectural decision eliminates an entire class of common testing problems and enables features such as real-time command execution, automatic waiting, time-travel debugging, and built-in screenshot and video capture.

4. What is Cypress?

Cypress is a free and open-source JavaScript testing framework designed to make it straightforward for developers and QA engineers to write reliable, maintainable automated tests for web applications. It supports all frameworks built on modern JavaScript, including React, Vue, Angular, and Next.js.

Attribute	Details
Full Name	Cypress.io
License	Open Source — MIT License (Free to use commercially)
Primary Language	JavaScript / TypeScript
Supported Frameworks	React, Vue, Angular, Next.js, Svelte, and any modern web app
Test Runner Modes	Interactive GUI (Test Runner) and Headless CLI (for CI/CD pipelines)
Browser Support	Chrome, Firefox, Edge, Electron (built-in)
Notable Users	GitHub, Shopify, NASA, Twitch, and thousands of enterprises worldwide
Repository	github.com/cypress-io/cypress (over 47,000 GitHub stars)

4.1 How Cypress Executes Tests

Cypress follows a sequential, declarative execution model. When a test runs, Cypress controls the browser directly and executes each command in the order it was written, automatically waiting for the application to settle before proceeding to the next step.

```
Step 1: Developer writes test in JavaScript
      ↓
Step 2: Cypress opens a real browser (Chrome / Firefox / Edge)
      ↓
Step 3: Cypress controls the browser — navigates, clicks, types
      ↓
Step 4: Cypress asserts expected vs. actual application state
      ↓
Step 5: Results reported — Pass ✓ or Fail X — with screenshot + video
```

5. Project Overview

5.1 Demo Application — Student Attendance Management System

To demonstrate the full capability of Cypress across multiple testing paradigms, a complete web application was built from scratch using Next.js 14. The application replicates a realistic academic attendance management scenario with two distinct user roles: students and administrators.

Page / Route	Description	Role
/login	Authentication page with form validation	All Users
/dashboard	Student view — mark attendance as Present or Absent	Student
/admin	Admin view — manage all student attendance records	Admin
/students	ClassPulse live roster — real-time student list with status toggles	All Users
/api/students	RESTful backend API endpoint — GET and POST operations	System

5.2 Application Credentials

Account Type	Email Address	Password	Redirect After Login
Student	student@test.com	123456	/dashboard
Administrator	admin@test.com	123456	/admin

5.3 Three Types of Testing Employed

The Cypress test suite for this project covers three distinct layers of the application stack, ensuring comprehensive coverage from the API layer through to the user interface and individual component behaviour.

Testing Type	Files	What It Targets	Tests
End-to-End (E2E)	demo.cy.js, attendance.cy.js	Full user journeys through the browser — navigation, forms, redirects, toasts	8
API Testing	api.cy.js	HTTP requests and responses to/from the backend REST API	3

Testing Type	Files	What It Targets	Tests
Component Testing	StudentItem.cy.js	An individual React component mounted in isolation	3
Total	4 files	Full-stack coverage	14

6. Core Cypress Commands Used

The following table documents every significant Cypress command utilised across the four test files. Understanding these commands is fundamental to understanding how the tests interact with the application under test.

Command	Purpose	Used In
cy.visit('/path')	Navigates the browser to the specified URL	All E2E files
cy.get('[data-cy=x]')	Selects a DOM element using a data-cy attribute — test-specific, resilient to CSS changes	demo.cy.js
cy.get('[data-testid=x]')	Selects a DOM element using a data-testid attribute	attendance.cy.js
cy.type('text')	Simulates keyboard input into a focused input element	Login tests
cy.click()	Simulates a mouse click event on the targeted element	All test files
cy.contains('text')	Locates an element that contains the specified text string	Toast & error checks
.should('be.visible')	Asserts that the element is currently visible in the viewport	Error messages
.should('have.text', 'x')	Asserts the element's exact text content matches the given string	Stat counters
.should('have.class', 'x')	Asserts a CSS class is present on the element	Active button state
.should('be.disabled')	Asserts that the element is in a disabled state	Post-attendance buttons
cy.request(method, url)	Issues a direct HTTP request — bypasses the UI entirely	api.cy.js
cy.mount(<Component />)	Renders a React component in isolation within Cypress	StudentItem.cy.js
cy.spy(obj, 'method')	Wraps a function to monitor calls, arguments, and return values	Component test 3
cy.clearLocalStorage()	Clears all browser localStorage data before each test run	beforeEach hooks
cy.window()	Returns the browser's window object for accessing localStorage or globals	Admin seed test

7. Detailed Test Analysis

7.1 demo.cy.js — Roster UI Tests

Test Type	Coverage
End-to-End (E2E)	ClassPulse Roster (/students page)

This file tests the ClassPulse live roster component — the most interactive page in the application. Tests verify DOM rendering, state management, and real-time counter updates, all without any page reload.

Test #	Test Name	Assertion Detail
1	Renders the dashboard correctly	Visits /students; locates the ClassPulse title via data-cy attribute; asserts exactly 3 student list items are rendered in the DOM at load time
2	Stat counters are accurate	Reads three independent counter elements — Total Students, Present, Absent — and asserts exact values of 3, 2, and 1 respectively
3	Adding a student updates the list	Types 'David Beckham' into the name input and presses Enter; asserts the list length becomes 4, new student appears at index 0, Total counter updates to 4, Present counter updates to 3
4	Status toggle updates counters and UI	Clicks the Absent button on Alice Johnson; asserts Present counter decreases from 2 to 1, Absent counter increases from 1 to 2, and the CSS active class switches from the Present to the Absent button

7.2 attendance.cy.js — Authentication & Full Flow Tests

Test Type	Coverage
End-to-End (E2E)	Login, Dashboard, Admin flows

This file tests complete user journeys — from unauthenticated access through login, attendance actions, and role-based redirection. It covers both the happy path and error scenarios for a realistic test suite.

Test #	Test Name	Assertion Detail
1	Empty form submission shows validation error	Clicks the Login button without entering any credentials; asserts the error message 'Please fill in both fields' appears and is visible

Test #	Test Name	Assertion Detail
2	Invalid credentials show error message	Enters incorrect email and password; waits for the loading spinner to resolve; asserts the error message 'Invalid credentials' appears
3	Full student attendance flow	Executes the complete journey: visit /login, fill form, submit, assert URL changes to /dashboard, verify navbar shows 'Hello, Student', click Mark Present, assert green success toast appears, assert both buttons become disabled, click Logout, assert redirect back to /login
4	Admin can override attendance record	Seeds a fake Absent record into browser localStorage using cy.window(); logs in as admin; locates the student row in the attendance table; clicks Mark Present; asserts the red Absent badge is removed and a green Present badge replaces it

7.3 api.cy.js — Backend API Tests

Test Type API Testing (HTTP)	Coverage /api/students — GET, POST, Error handling
---------------------------------	---

About API Testing in Cypress

Cypress is not limited to browser-based UI testing. Using `cy.request()`, Cypress can send real HTTP requests directly to backend endpoints — functioning similarly to Postman or curl, but integrated natively within the test suite. This allows the backend API to be verified independently of the frontend interface.

Test #	Test Name	Assertion Detail
1	GET /api/students returns correct data	Sends an HTTP GET request to /api/students; asserts HTTP status 200, response body contains success: true, the data property is an array, and the first student's name is 'Emma Watson'
2	POST /api/students creates a new student	Sends an HTTP POST request with body { name: 'Will Smith', status: 'present' }; asserts HTTP status 201 Created, response body contains the submitted name, and a server-generated id property exists on the returned object
3	POST with missing name field returns 400 error	Sends a POST request with the name field omitted; uses failOnStatusCode: false to prevent Cypress from treating the error as a test failure; asserts exactly HTTP status 400 and the error message 'Name is required' in the response body

7.4 StudentItem.cy.js — React Component Tests

Test Type Component Testing	Coverage StudentItem React component — isolated rendering
--------------------------------	--

About Component Testing in Cypress

Component testing is Cypress's third mode of operation. Rather than navigating to a URL, `cy.mount()` renders a React component in a completely isolated environment within the Cypress runner. This allows testing a component's rendering logic, prop handling, CSS class application, and event callbacks without launching the full application.

Test #	Test Name	Assertion Detail
1	Component renders correctly	Mounts StudentItem with the student name 'Zendaya Coleman'; asserts the name text is displayed correctly; asserts the avatar element shows the auto-generated initials 'ZC' derived from the student's name

Test #	Test Name	Assertion Detail
2	Active button state reflects student status	Mounts StudentItem with status set to 'present'; asserts the Present button has the CSS class active; asserts the Absent button does not have the active class — confirming the visual state correctly reflects prop input
3	Click handler is called with correct arguments	Creates a cy.spy() function and passes it as the markStatus prop; clicks the Absent button; asserts the spy was called exactly once; asserts it was called with arguments ('77', 'absent') — proving the component's click handler invokes the callback with the correct student ID and status

8. Key Technical Concepts

The following technical concepts underpin the design decisions in this test suite. Understanding these principles is essential for writing robust, maintainable tests at a professional standard.

8.1 Test Isolation with `beforeEach()`

Every test in this suite begins with a `beforeEach()` hook that calls `cy.clearLocalStorage()`. This ensures each test begins from an identical, clean state — completely independent of any other test. This practice, known as test isolation, is a fundamental requirement of reliable automated testing. Without it, a test that fails may corrupt state and cause subsequent, otherwise passing tests to fail, producing misleading results.

8.2 Semantic Test Selectors (`data-cy` / `data-testid`)

Rather than selecting elements by CSS class names or IDs — which are subject to change during UI refactoring — this test suite uses dedicated `data-cy` and `data-testid` HTML attributes. These attributes exist exclusively for testing purposes and carry no styling or functional implications. This makes tests resilient to design changes: a developer can freely rename CSS classes without ever breaking a test.

8.3 `cy.spy()` — Behaviour Verification

A spy is a special type of function wrapper that records every call made to it, including the arguments passed and the number of invocations. In test file `StudentItem.cy.js`, `cy.spy()` is used to verify that clicking the Absent button causes the component to invoke the `markStatus` callback with precisely the correct arguments. This technique allows testing the internal logical behaviour of a component independently of the visual outcome.

8.4 `failOnStatusCode: false` — Testing Error Paths

By default, Cypress treats any HTTP response with a 4xx or 5xx status code as a test failure, immediately stopping test execution. Setting `failOnStatusCode: false` overrides this behaviour, allowing the test to receive the error response and make assertions against it. This is critical for verifying that error-handling logic in the backend is working correctly — not just the happy path.

8.5 Custom Commands — The DRY Principle

The custom `cy.loginAs(role)` command, defined in `cypress/support/commands.js`, encapsulates the entire login sequence — navigating to `/login`, filling the email and password fields, and clicking the submit button — into a single reusable command. This applies the Don't Repeat Yourself (DRY) software engineering principle to test code: the login logic is maintained in one location, and a change to the login flow requires updating only the custom command, not every test that performs a login.

8.6 `cy.window()` — Accessing Browser Internals

Cypress runs its own JavaScript in a Node.js process, separate from the browser window. Direct access to `window.localStorage` from within a test therefore does not reference the browser's storage — it references Node's environment. `cy.window()` is the correct bridge: it returns a reference to the actual browser window object, allowing tests to read and write to `localStorage`, access global variables, or call window-level functions within the application under test.

9. Results & Discussion

9.1 Test Results Summary

File	Testing Type	Tests Written	Tests Passing	Pass Rate
demo.cy.js	End-to-End — UI	4	4	100%
attendance.cy.js	End-to-End — Auth & Flows	4	4	100%
api.cy.js	API (HTTP)	3	3	100%
StudentItem.cy.js	React Component	3	3	100%
Overall	3 Methodologies	14	14	100%

9.2 Discussion

All fourteen tests pass successfully, providing full confidence across the three layers of the application stack. Several observations are worth highlighting from the implementation:

- State management verification was the most technically demanding aspect of the E2E tests. Testing that React state updates — such as counter increments after adding a student — correctly reflected in the DOM required precise use of `.should()` assertions with retry logic built into Cypress's automatic waiting mechanism.
- API testing proved significantly faster than E2E testing, as it bypasses the browser rendering entirely. The three API tests complete in under two seconds, compared to several seconds for the full E2E flows.
- Component testing provided the highest degree of precision for unit-level behaviour. Asserting that the `markStatus` spy was called with `('77', 'absent')` would be impossible to verify through E2E testing alone, demonstrating the complementary value of all three testing methodologies working together.
- The `cy.window()` technique used in the Admin Override test (Test 4 of `attendance.cy.js`) demonstrates an advanced Cypress pattern: programmatic state seeding without UI interaction. This is a recommended best practice for testing scenarios that require pre-existing data, as it avoids slow UI-based setup steps.

Key Takeaway

The three testing methodologies — E2E, API, and Component — are not competing approaches. They are complementary layers that, used together, provide comprehensive coverage: E2E tests verify the user experience, API tests verify backend correctness, and component tests verify individual building blocks. A mature test suite employs all three.

10. Conclusion

This presentation has demonstrated Cypress as a mature, production-grade open-source testing framework that substantially raises the quality bar for web application development. Through the development of a real-world Student Attendance Management System and the authoring of fourteen automated tests, the following conclusions can be drawn:

- **Accessibility:** Cypress simplifies testing without sacrificing power. Its JavaScript-native syntax, automatic waiting, and real browser execution eliminate the complexity and flakiness historically associated with E2E testing tools. Developers who already know JavaScript face virtually no learning curve.
- **Unified Tooling:** Multi-layer coverage is achievable within a single tool. The ability to write E2E tests, API tests, and component tests within the same framework — using a consistent API — reduces tooling overhead and allows teams to maintain a single test configuration.
- **Best Practices:** Test design principles matter as much as the framework itself. Concepts such as test isolation, semantic selectors, the DRY principle, and spy-based behaviour verification are framework-agnostic best practices that translate to any testing environment. Cypress provides excellent affordances for applying these principles correctly.
- **Recommendation:** Cypress is an appropriate choice for any team building modern JavaScript web applications. Its active open-source community, comprehensive documentation, and free licensing make it accessible to teams of all sizes — from solo developers to large engineering organisations.

In summary, Cypress delivers on its promise of making testing a first-class, developer-friendly activity. The combination of its intuitive API, powerful debugging tools, and multi-paradigm testing support makes it one of the most compelling open-source contributions to the web development ecosystem in the past decade.

Project Outcome

14 tests written. 14 tests passing. 3 testing methodologies demonstrated. 1 complete, working web application built and tested from scratch. All tests execute with 100% pass rate across End-to-End, API, and Component testing layers.

11. References

The following sources were consulted in the preparation of this presentation and the development of the demonstration application:

- [1] Cypress.io Official Documentation. *Cypress — Getting Started Guide*. Available at: <https://docs.cypress.io/guides/overview/why-cypress>
- [2] Cypress.io. *Component Testing Introduction*. Available at: <https://docs.cypress.io/guides/component-testing/overview>
- [3] Cypress.io. *API Testing with cy.request()*. Available at: <https://docs.cypress.io/api/commands/request>
- [4] Next.js by Vercel. *Next.js 14 Documentation*. Available at: <https://nextjs.org/docs>
- [5] Martin Fowler. *The Practical Test Pyramid*. Available at: <https://martinfowler.com/articles/practical-test-pyramid.html>
- [6] Kent C. Dodds. *Static vs Unit vs Integration vs E2E Testing*. Available at: <https://kentcdodds.com/blog/testing-trophy>
- [7] Cypress GitHub Repository. *cypress-io/cypress*. Available at: <https://github.com/cypress-io/cypress>

END OF DOCUMENT

Ashley Almeida | Jaden Britto | Naomi Countino

Open Source Tool Presentation — Cypress